

Lecture 21: Structured Streaming III (Practical)

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

April 9, 2026

1. Streaming Sources
2. Streaming Sinks
3. Output Modes & Triggers
4. Structured Streaming Joins
5. Deduplication

Streaming Sources

Available Sources in Spark

- **File Source:** Reads files written in a directory as a stream. Supported formats: text, csv, json, orc, parquet.
- **Kafka Source:** Reads data from Apache Kafka.
- **Socket Source (Testing only):** Reads UTF-8 text data from a socket connection.
- **Rate Source (Testing only):** Generates data at a specified number of rows per second.

1. Socket Source

- Primarily used for learning and debugging.
- Does not provide fault-tolerance (offsets are not tracked).

Scala

```
1 val lines = spark.readStream
2   .format("socket")
3   .option("host", "localhost")
4   .option("port", 9999)
5   .load()
6
```

Python

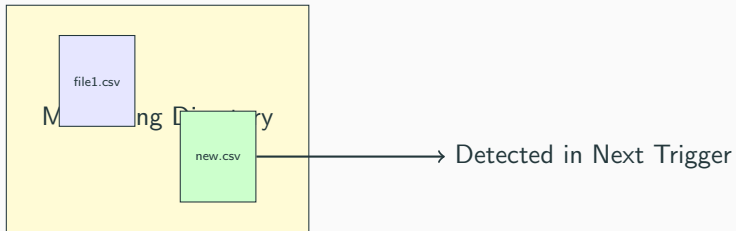
```
1 lines = spark \
2     .readStream \
3     .format("socket") \
4     .option("host", "localhost") \
5     .option("port", 9999) \
6     .load()
7
```

2. File Source

- Spark monitors a directory for new files.
- Files must be atomically moved into the directory (e.g., mv not cp).
- **Key Option:** `maxFilesPerTrigger` - controls the batch size.

```
1 userSchema = StructType().add("name", "string").add("age", "integer")
2 csvDF = spark \
3   .readStream \
4   .option("sep", ";") \
5   .schema(userSchema) \
6   .csv("/path/to/directory") # Equivalent to .format("csv").load("/path/...")
7
```

File Source Internals



Spark maintains a log of processed files to avoid re-reading.

3. Kafka Source

- The industry standard for production streaming.
- Requires spark-sql-kafka-0-10 artifact.
- Supports reading from multiple topics and partitions.

```
1 df = spark \  
2   .readStream \  
3   .format("kafka") \  
4   .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \  
5   .option("subscribe", "topic1") \  
6   .load()  
7
```


Kafka Configuration Options

- **startingOffsets**: earliest, latest, or a JSON string `{"topic1":{"0":23}}`.
- **failOnDataLoss**: Whether to fail the query when it's possible that data is lost (e.g., Kafka retention policy kicks in). Default is `true`.
- **maxOffsetsPerTrigger**: Limits the number of offsets processed per trigger (Backpressure).

Schema of Kafka Data

The Kafka source provides a DataFrame with a fixed schema:

Column	Type	Description
key	binary	The key of the message
value	binary	The payload of the message
topic	string	Topic name
partition	int	Partition number
offset	long	Offset in partition
timestamp	timestamp	Timestamp of message (event time)

Parsing Kafka JSON Data (Python)

```
1 from pyspark.sql.functions import from_json, col
2
3 schema = StructType().add("id", "integer").add("data", "string")
4
5 parsedDF = df.selectExpr("CAST(value AS STRING)" \
6     .select(from_json(col("value"), schema).alias("data")) \
7     .select("data.*"))
```

Streaming Sinks

Output Sinks Overview

- **File Sink:** Stores output to a directory (Parquet, ORC, etc.).
- **Kafka Sink:** Writes output to a Kafka topic.
- **Console Sink:** Prints output to the console (for debugging).
- **Memory Sink:** Stores output in memory as a table (for small scale debugging).
- **Foreach/ForeachBatch:** Arbitrary logic on every row/batch.

1. File Sink Details

- Supports partitioning and bucketing.
- Requires a **checkpoint location**.
- Guaranteed exactly-once if the sink is file-based and used with a WAL.

```
1 query = df.writeStream \  
2   .format("parquet") \  
3   .option("path", "path/to/destination") \  
4   .option("checkpointLocation", "path/to/checkpoint") \  
5   .partitionBy("year", "month") \  
6   .start()  
7
```

2. Kafka Sink Details

- Write results back to Kafka for downstream consumption.
- Columns required: value (binary/string).
- Columns optional: key, topic, partition.

```
1 query = df.selectExpr("CAST(id AS STRING) AS key", "to_json(struct(*)) AS value") \  
2   .writeStream \  
3   .format("kafka") \  
4   .option("kafka.bootstrap.servers", "host1:port1") \  
5   .option("topic", "output-topic") \  
6   .start()  
7
```

3. Console and Memory Sinks

- **Console:** Displays top 20 rows.

```
1 df.writeStream.format("console").option("truncate", "false").start()  
2
```

- **Memory:** For testing only.

```
1 df.writeStream \  
2   .format("memory") \  
3   .queryName("my_results") \  
4   .start()  
5 # Query later: spark.sql("SELECT * FROM my_results").show()  
6
```


4. Foreach Sink

- Used for custom row-level logic.
- Must implement `open()`, `process()`, and `close()` methods.
- Harder to use than `foreachBatch` but more granular.

5. ForeachBatch Sink: Power and Flexibility

- Reuse batch data sources.
- Write to multiple sinks in one go.
- Apply batch-only optimizations (e.g. `persist()`) within the function.

```
1 def multi_sink(batch_df, batch_id):  
2     batch_df.persist()  
3     batch_df.write.format("jdbc").save(...) # To DB  
4     batch_df.write.format("parquet").save(...) # To Data Lake  
5     batch_df.unpersist()  
6  
7 df.writeStream.foreachBatch(multi_sink).start()  
8
```

Output Modes & Triggers

The Three Output Modes

Append Mode

✓ Only new rows since last trigger

Update Mode

✓ Only changed rows since last trigger

Complete Mode

✓ The whole table every time

Mode Comparison: Aggregation Case

Suppose we are counting words.

- **Complete:** Output: (A, 1), (B, 1) in Batch 1. Output: (A, 2), (B, 1), (C, 1) in Batch 2.
- **Update:** Output: (A, 1), (B, 1) in Batch 1. Output: (A, 2), (C, 1) in Batch 2.
- **Append:** Requires Watermark. Output: nothing until watermark closes the window.

Understanding Triggers

Triggers define *when* data is processed.

- **Default:** Process as soon as previous micro-batch finishes.
- **ProcessingTime:** Fixed intervals (e.g., 10 seconds).
- **Once:** Process all available data once and stop (Batch-like).
- **Continuous:** Ultra-low latency (Experimental).

Trigger Code Examples

Scala

```
1 import org.apache.spark.sql.streaming.Trigger
2
3 df.writeStream
4   .trigger(Trigger.ProcessingTime("10 seconds"))
5   .start()
6
7 df.writeStream
8   .trigger(Trigger.Once())
9   .start()
```

Python

```
1 df.writeStream \
2   .trigger(processingTime='10 seconds') \
3   .start()
4
5 df.writeStream \
6   .trigger(once=True) \
7   .start()
```

Structured Streaming Joins

Types of Joins

- **Stream-Static Join:** Joining a stream with a static DataFrame (e.g., enriching logs with user metadata).
- **Stream-Stream Join:** Joining two streaming DataFrames (e.g., joining ad impressions with clicks).

1. Stream-Static Join

- Very efficient. Spark treats the static DF as a lookup table.
- Static DF is not updated automatically when the source file changes (needs restart).

```
1 staticDF = spark.read.parquet("/user/metadata")
2 streamingDF = spark.readStream.kafka(...)
3
4 enrichedDF = streamingDF.join(staticDF, "userId")
5
```

2. Stream-Stream Join (Inner)

- Challenging: Spark must buffer data from both streams in its state.
- To avoid infinite state growth, you **must** use:
 1. Watermarks on both inputs.
 2. Constraints on event-time (e.g., `clickTime < impressionTime + 1h`).

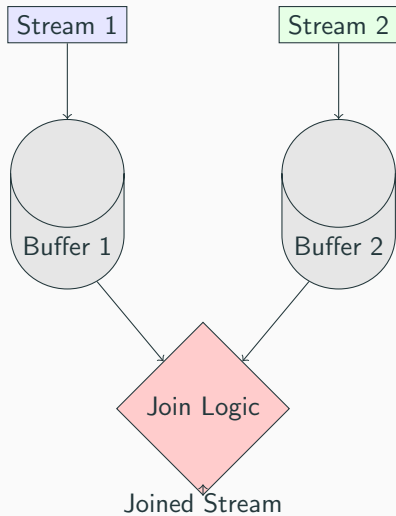
Stream-Stream Inner Join Example (Scala)

```
1 val impressions = spark.readStream. ...  
2   .withWatermark("impressionTime", "2 hours")  
3  
4 val clicks = spark.readStream. ...  
5   .withWatermark("clickTime", "3 hours")  
6  
7 impressions.join(clicks,  
8   expr("""  
9     clickAdId = impressionAdId AND  
10    clickTime >= impressionTime AND  
11    clickTime <= impressionTime + interval 1 hour  
12    """)  
13 )
```

3. Stream-Stream Outer Join

- Supports Left, Right, and Full Outer joins.
- Watermarking and time constraints are **mandatory**.
- Outer results are only emitted once the watermark passes the time constraint (to ensure no more matches are possible).

Stream-Stream Join Architecture



Deduplication

Streaming Deduplication

- Use `dropDuplicates()` on a streaming `DataFrame`.
- Spark maintains the state of seen keys.
- Use Watermarking to expire old keys from the state.

Deduplication Code

```
1 # Deduplicate using 'guid' column, but only keep state for 1 hour
2 df.withWatermark("timestamp", "1 hour") \
3   .dropDuplicates(["guid"])
```

Questions?